

INSTITUTO SAN IGNACIO DE LOYOLA – ISIL

TÍTULO:
Proceso de Aprendizaje 3

CURSO:
Programación Web Avanzada

NRC:
4057

DOCENTE:
Rebaza Chumacero Jorge Eduardo

ALUMNOS:

Corrales Granda Lucero Helen
Samame Rivera Sol Fernanda
Gutierrez Perez Yanhira
Oganeku Fernandez Franco Rodrigo

CICLO:
2026 - 2

NexoTech

Informe técnico de autenticación, autorización y panel React

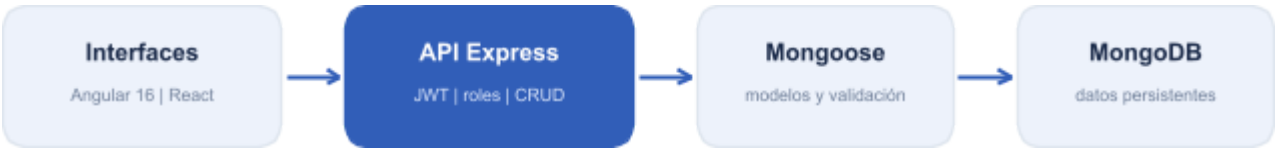
Sistema de gestión de productos electrónicos con Angular 16, React, Express y MongoDB.

PROYECTO	ESTADO	FECHA
Catálogo NexoTech	Implementado y compilado localmente	4 de noviembre de 2026

Resumen ejecutivo

La entrega amplía el CRUD electrónico de PA2 con cuentas de usuario, contraseñas protegidas con bcrypt, tokens JWT con expiración y permisos por rol. Angular 16 continúa como interfaz principal y un panel React alternativo consume la misma API Express y la misma base MongoDB.

Diseño de alto nivel



Las dos interfaces comparten autenticación, API y catálogo de MongoDB.

Resultado local: 9 pruebas automatizadas aprobadas y builds de producción de Angular y React completados. El despliegue externo queda preparado mediante render.yaml; no se generó una URL pública.

1. Arquitectura y datos

El backend mantiene la estructura MVC existente: las rutas seleccionan middleware y controladores; los controladores aplican reglas de negocio; Mongoose valida y persiste documentos en MongoDB. Express sirve el build Angular desde la raíz y el build React en /panel.

CAPA	RESPONSABILIDAD
Rutas	Declaran endpoints y aplican autenticación y permisos.
Controladores	Gestionan registro, acceso, sesión, roles y CRUD.
Middleware	Verifica firma, expiración, emisor, audiencia y versión de sesión JWT.
Modelos	Definen producto y usuario; el hash de contraseña queda excluido de lecturas ordinarias.
Clientes	Angular usa guards e interceptor; React usa Router, Context, Hooks y Axios.

Modelo de usuario

CAMPO	REGLA
nombre	Obligatorio, de 2 a 80 caracteres.
email	Obligatorio, único, normalizado en minúsculas y validado.
passwordHash	Hash bcrypt; no se devuelve al cliente.
rol	admin, editor o viewer.
tokenVersion	Contador para invalidar tokens al cerrar sesión o al cambiar el rol.

Permisos

ROL	PERMISOS
admin	Consulta y administra productos, usuarios y roles.
editor	Consulta, crea, edita y elimina productos.
viewer	Consulta productos.

2. API y experiencia de uso

La API usa JSON y devuelve mensajes de error legibles. El token se envía como Bearer en cada solicitud protegida. Las rutas de escritura validan los permisos del lado del servidor, incluso si el cliente no muestra sus botones.

MÉTODO	RUTA	ACCESO
POST	/api/auth/registro	Público, con límite de intentos.
POST	/api/auth/login	Público, con límite de intentos.
POST	/api/auth/logout	Usuario autenticado.
GET	/api/auth/me	Usuario autenticado.
GET	/api/auth/usuarios	Solo admin; lista cuentas.
PATCH	/api/auth/usuarios/:id/rol	Solo admin; actualiza un rol.
GET	/api/productos	Usuario autenticado.
POST / PUT / DELETE	/api/productos	Admin y editor.

Angular 16

Interfaz principal con catálogo y dashboard, formularios reactivos, validaciones, búsqueda y filtro por categoría. Guards funcionales protegen páginas privadas y de administración; el interceptor añade el JWT y lleva a acceso cuando el servidor devuelve 401.

React

Interfaz secundaria compilada bajo /panel. Incluye React Router DOM, componentes funcionales, Hooks, Context para sesión y catálogo compartido entre páginas, Axios, formularios CRUD, filtro por texto/categoría y administración de roles.

Comportamiento del registro

ADMIN_EMAIL designa la cuenta administradora inicial. Si se configura, el primer registro debe coincidir con ese correo; las cuentas nuevas reciben rol viewer. Si la variable está vacía, la primera cuenta creada recibe admin. El administrador cambia roles desde la vista Usuarios.

3. Instalación, verificación y despliegue

Requisitos

Node.js 20.19 o posterior, npm y una instancia local o Atlas de MongoDB. Mongoose 9 declara ese mínimo de Node.

Inicio local

En la raíz: npm run setup; copia .env.example a .env solo si todavía no existe; configura MONGO_URI, JWT_SECRET de al menos 32 caracteres y ADMIN_EMAIL; inicia MongoDB y ejecuta npm start. Express sirve Angular en la raíz y React en /panel. En desarrollo se ejecutan npm run api:dev, npm run frontend:dev y npm run panel:dev en terminales separadas.

Variables principales

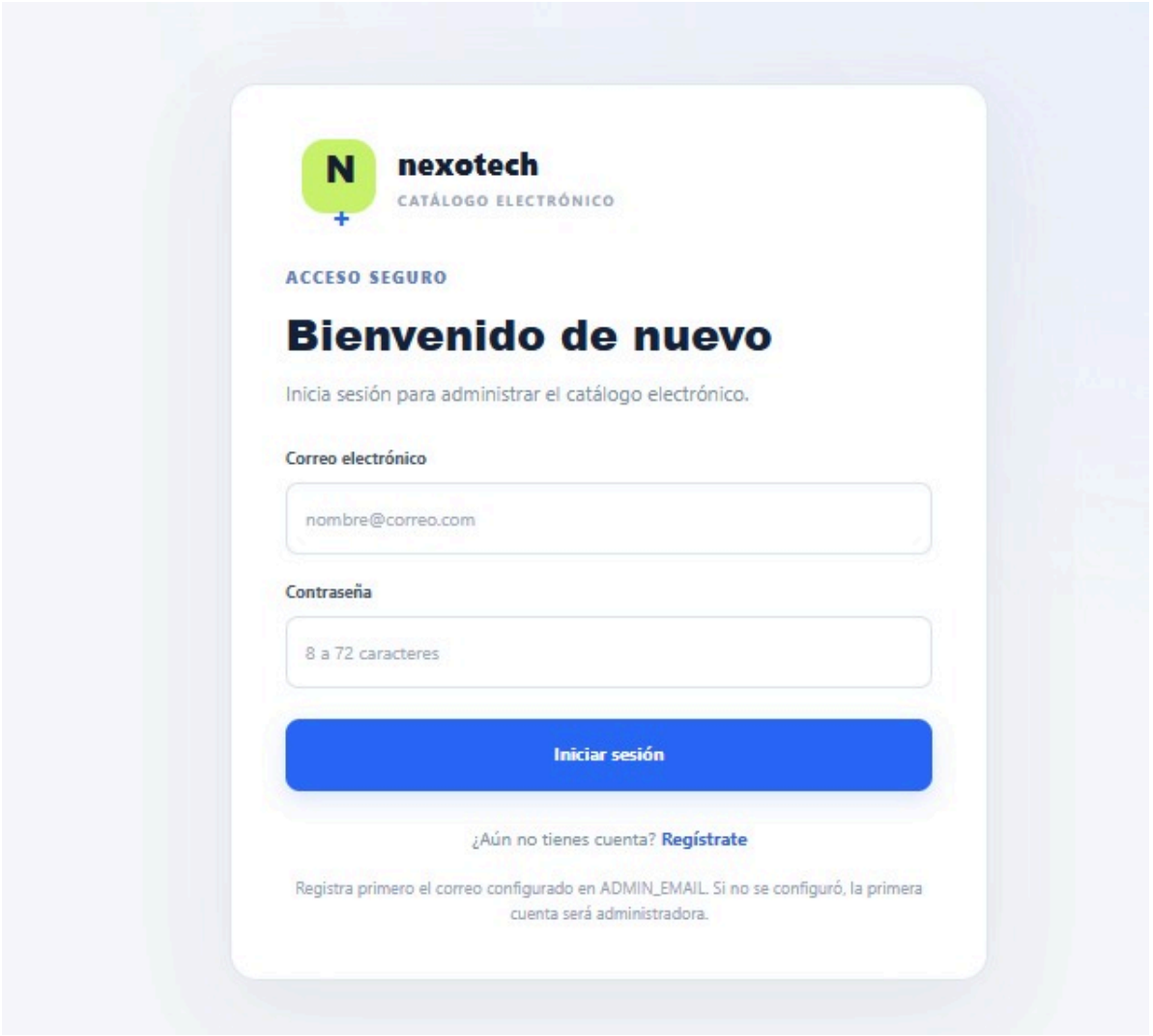
VARIABLE	USO
MONGO_URI	Conexión a MongoDB; se conserva fuera del repositorio.
JWT_SECRET	Secreto privado, generado localmente y nunca incluido en la entrega.
JWT_EXPIRES_IN	Duración del token; por defecto 8h.
ADMIN_EMAIL	Correo para el alta inicial del administrador.
CORS_ORIGINS	Orígenes de desarrollo separados por coma.

Verificación ejecutada

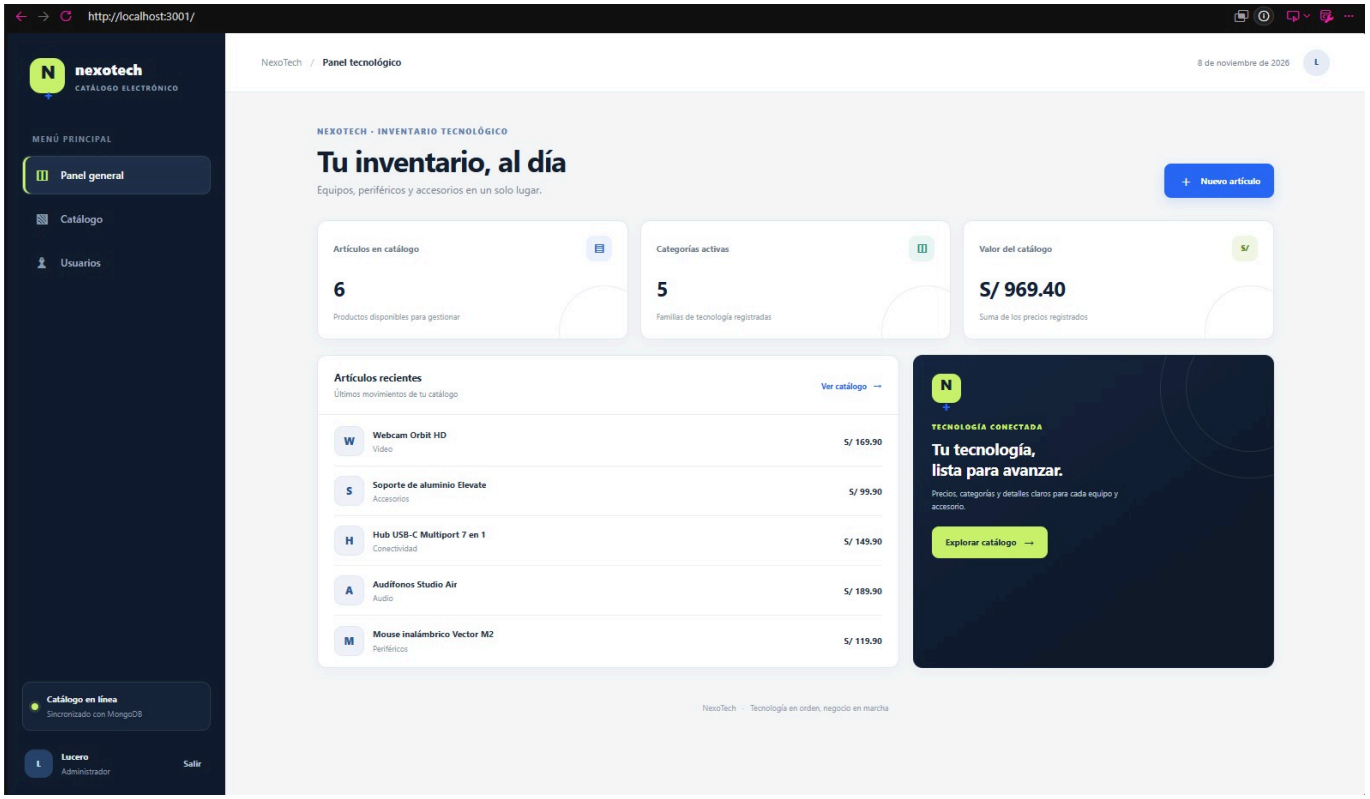
COMPROBACIÓN	RESULTADO
Pruebas automatizadas	9 aprobadas; 0 fallidas. Incluyen bcrypt, roles, claims JWT, registro, revocación y protección HTTP del CRUD.
Angular	Build de producción completado; salida en backend/public.
React	TypeScript y build de producción completados; salida en backend/public/panel.
Pruebas de flujo completo	Guía manual disponible en docs/manual-pruebas.md; requiere MongoDB aislado.

Despliegue

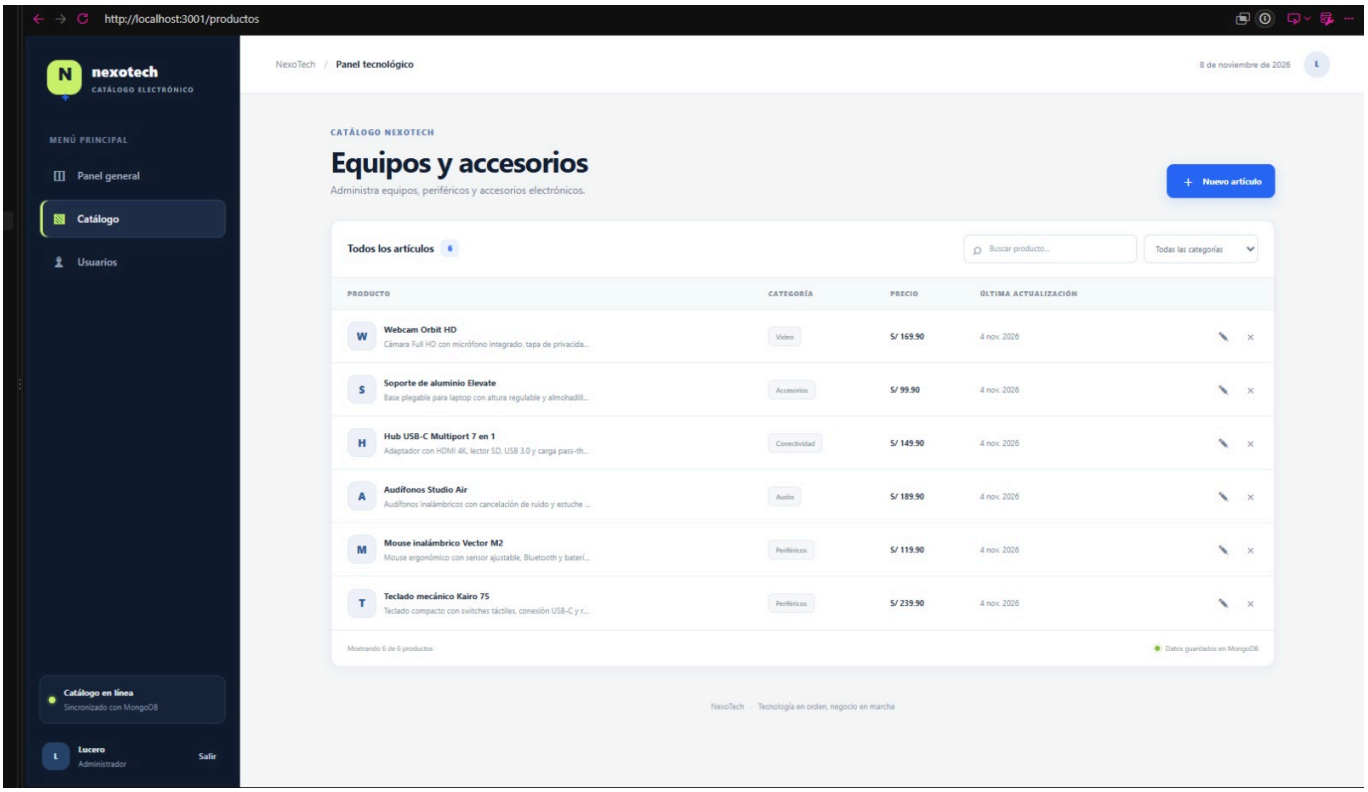
render.yaml contiene el servicio web, comandos de build/arranque y variables esperadas. No hay URL desplegada: el proyecto no tiene una cuenta de hosting ni una URI Atlas de entrega asociada. Para publicarlo, configura MONGO_URI y ADMIN_EMAIL en Render y despliega el repositorio.



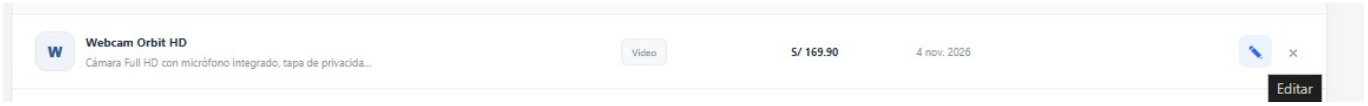
Permite ingresar al sistema mediante el correo y la contraseña para acceder de forma segura.



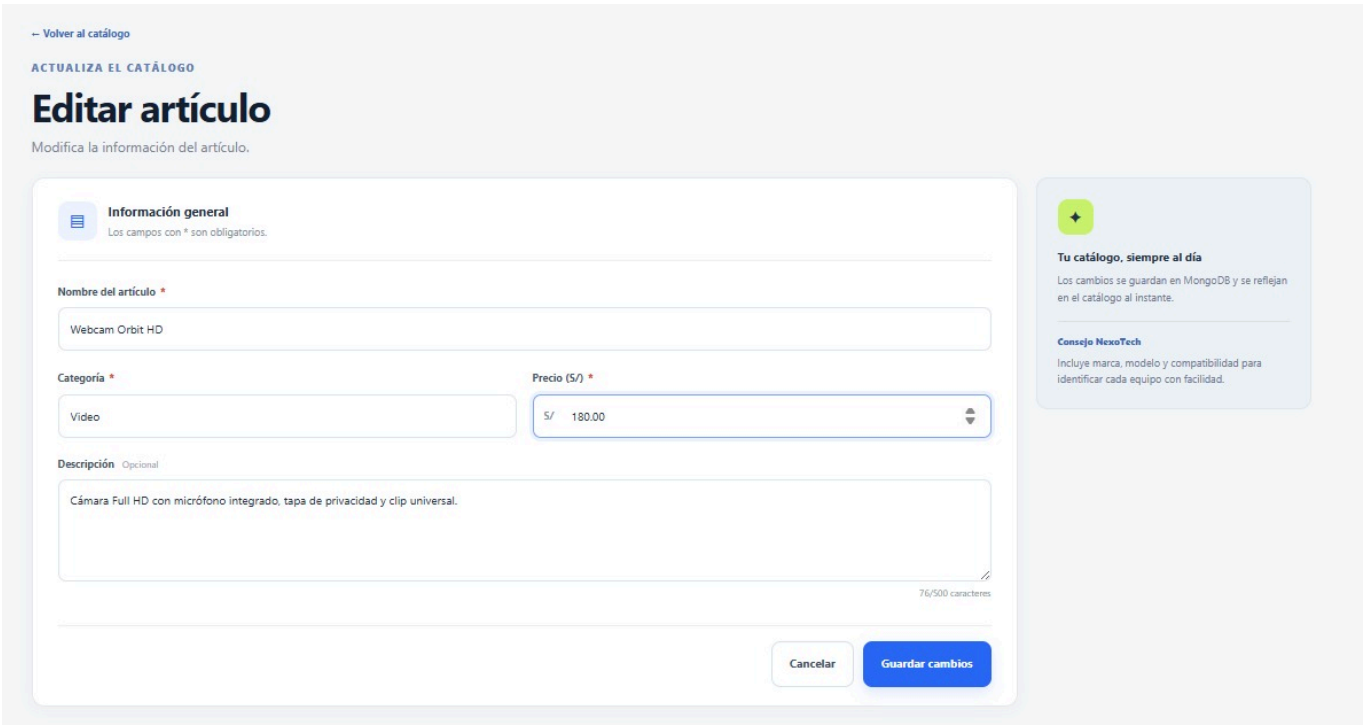
Permite visualizar el resumen de productos y acceder a las diferentes opciones de gestión.



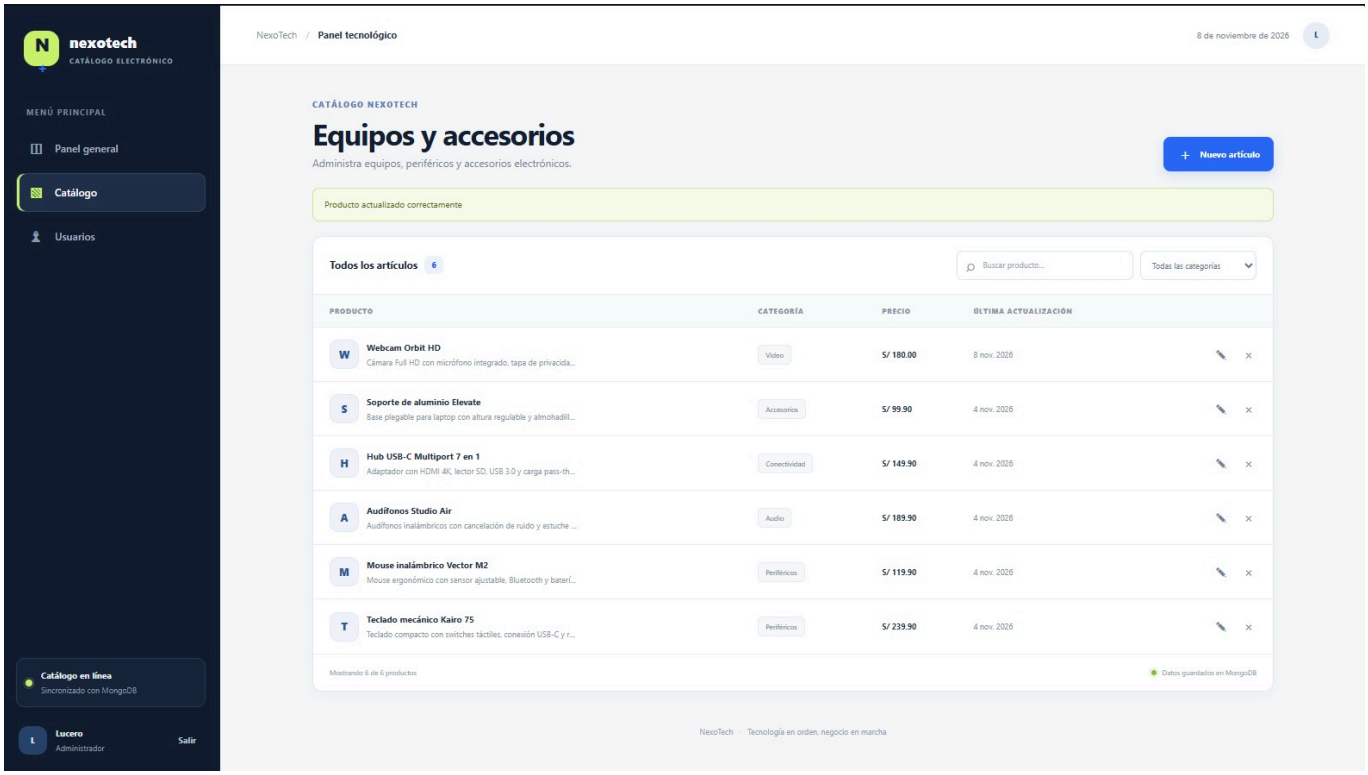
Permite consultar, buscar y administrar los productos registrados en el sistema.



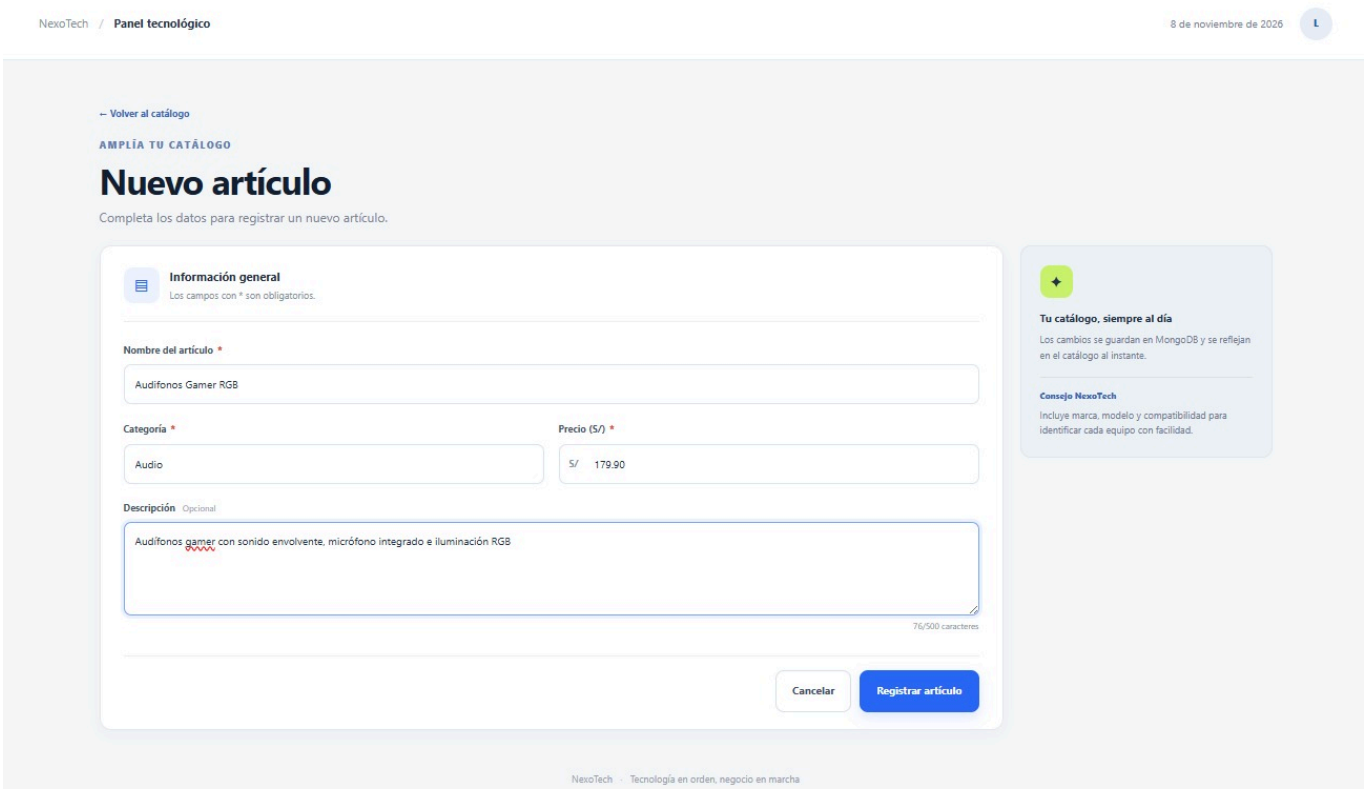
Permite consultar, buscar y administrar los productos registrados en el sistema.



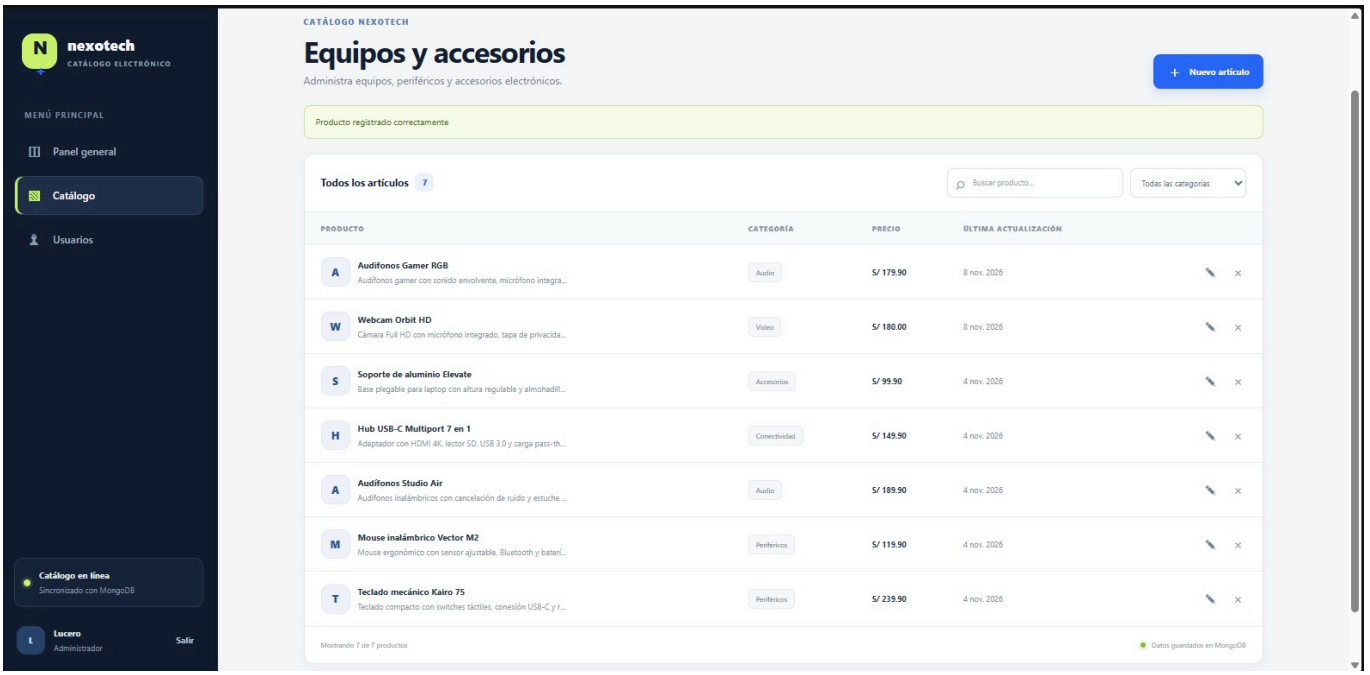
Permite modificar los datos de un artículo para mantener actualizada la información.



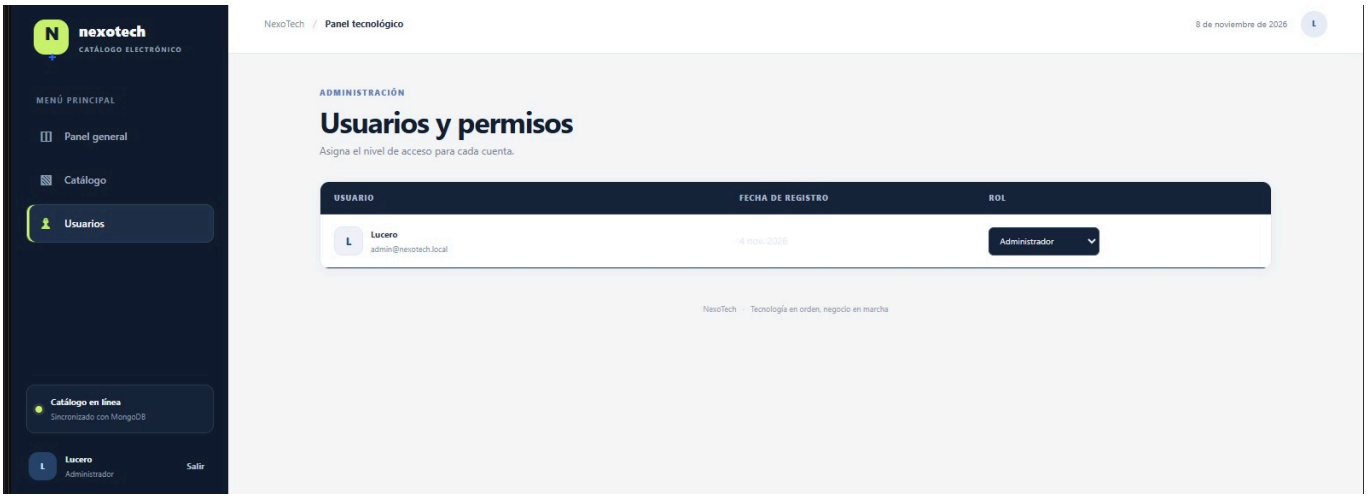
Permite verificar los cambios realizados y visualizar los productos actualizados.



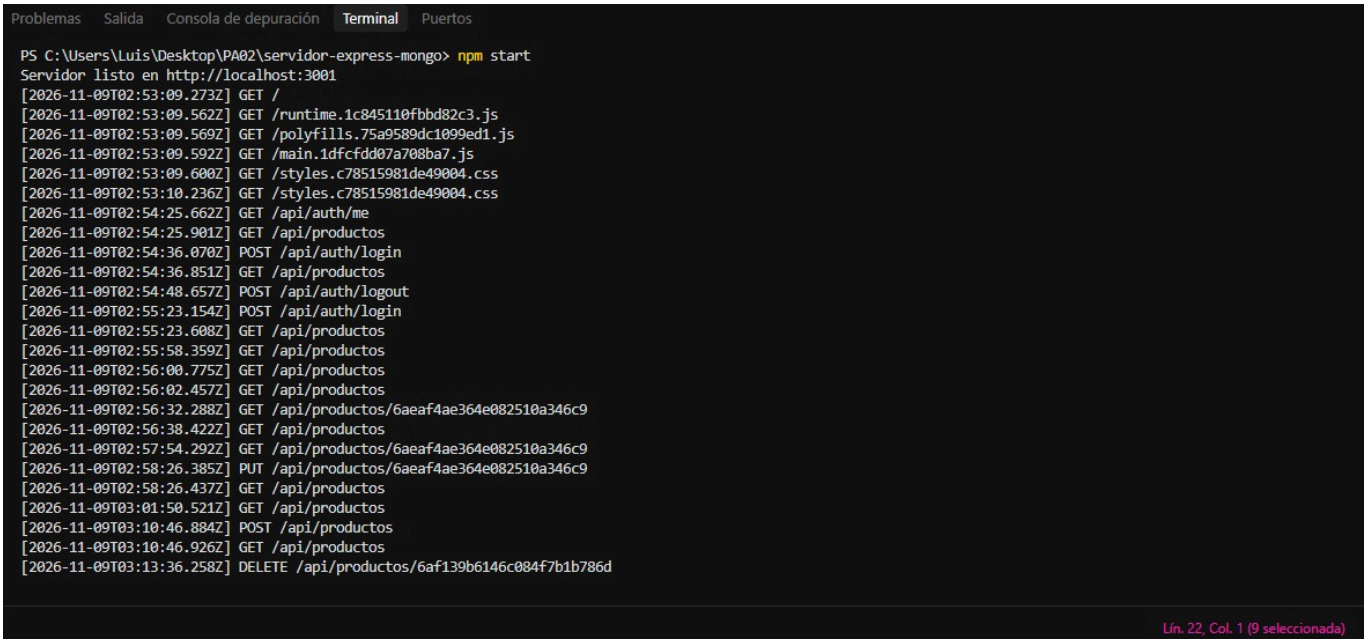
Permite ingresar nuevos artículos y almacenar sus datos en la base de datos.



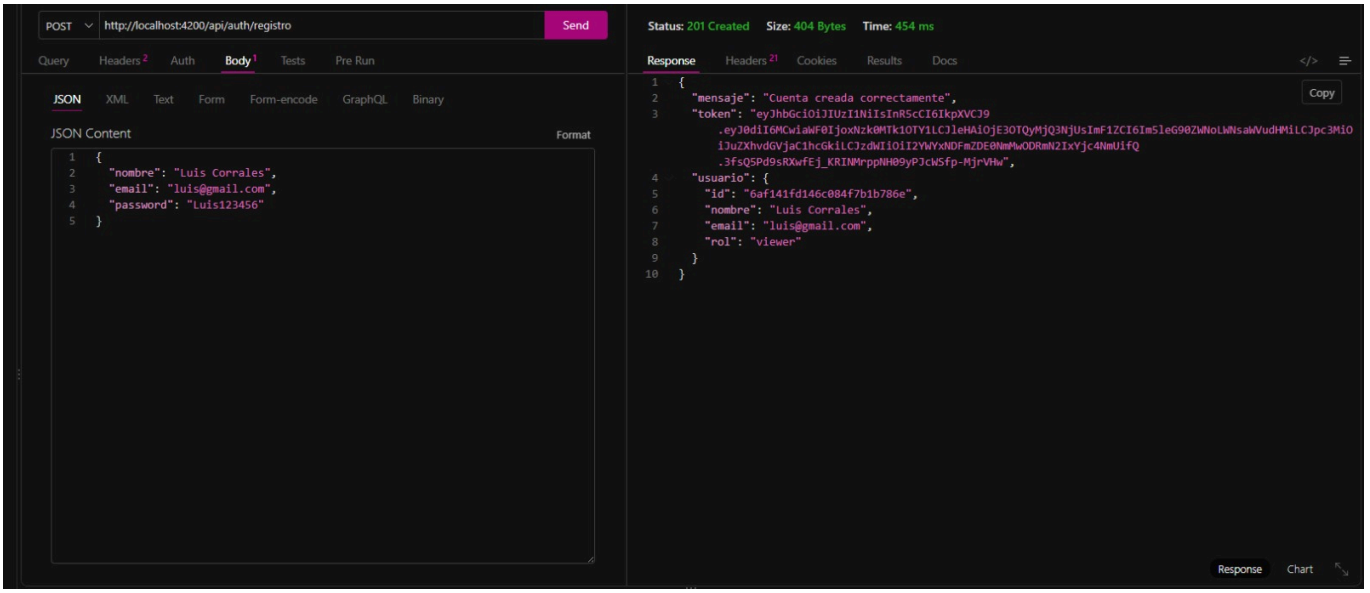
Permite visualizar los nuevos productos registrados y comprobar su incorporación.



Permite visualizar las cuentas registradas y cambiar los roles para controlar los permisos.



Permite ejecutar y revisar las solicitudes realizadas para comprobar el funcionamiento de la aplicación.



Permite enviar datos en formato JSON y recibir la respuesta del servidor para verificar el registro de usuarios.